

RESTAURANT APP MVP

Software Requirements Specification (SRS)

Frontend Only - React Native - Fall 2026

Assignment	Assignment No. 1
Student Name	Murad Khan
Registration No.	9256
Repository	restaurant-app-mvp

Prepared for academic submission

Question 1: Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the requirements for the Restaurant App MVP, a frontend-only React Native application. The document describes the expected behaviour of the customer and manager interfaces, local mock data, navigation, validation, state management, performance requirements, and the screens that will be implemented. The intended audience is the course instructor, student developer, and anyone reviewing or testing the prototype. This SRS covers version 1.0 of the MVP.

1.2 Scope

The Restaurant App MVP is designed to let customers browse a restaurant menu, search and filter items, manage a cart, apply mock promo codes, reserve a table, place a dine-in or takeaway order, and track the order status. Restaurant managers can view incoming orders, accept or decline reservations, and manage menu items. The application is implemented only on the client side.

- Customer login and signup using local mock users.
- Role-based navigation for Customer and Manager.
- Menu browsing with categories, daily specials, availability, search, sorting, favourites, and pull-to-refresh.
- Cart management with quantities, notes, promo codes, live totals, and item count badge.
- Table reservation using mock table and reservation data.
- Order placement for dine-in and takeaway.
- Order tracking with demo timers and status progression.
- Manager dashboard for orders, reservations, and menu management.
- Light and dark theme.
- Local persistence of orders, reservations, and menu edits using AsyncStorage.

Out of scope:

- Real online payments or payment gateway integration.
- Push notifications.
- Real backend server, database, cloud authentication, or external API.
- Real-time multi-device synchronization.
- Production security, encryption, and deployment infrastructure.

1.3 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification; a document that defines what the system must do.
MVP	Minimum Viable Product; the smallest working version that demonstrates the required features.
UML	Unified Modeling Language; diagrams used to model actors, classes, flows, states, and components.
Hook	A React function such as <code>useState</code> or <code>useEffect</code> that adds React features to functional components.
Context	React Context API used to share data across components without passing props through every level.
Reducer	A pure function that receives current state and an action and returns a new state.
Mock Data	Local sample data used instead of a backend database or API.

FlatList	A React Native component optimized for rendering scrollable lists.
AsyncStorage	Local key-value storage used to persist selected app data on the device.
Debounce	A technique that delays an action until the user stops typing for a specified period.

2. Overall Description

2.1 User Roles

Role	Key Permissions & Goals
Customer / Diner	Create an account, log in, browse/search menu, add items to cart, apply promo code, reserve a table, place an order, track status, manage favourites, view profile, switch theme, and logout.
Restaurant Manager	Log in, access the manager-only dashboard, view and update incoming orders, accept/decline reservations, add menu items, edit prices, toggle availability, view profile, switch theme, and logout.

2.2 Customer Self-Interview

As a potential customer, I would download the app if it saves time, clearly shows prices and availability, helps me find food quickly, allows me to reserve a table before arriving, makes the bill clear before ordering, and lets me track whether my order is pending, preparing, ready, or served. I would also value a dark mode, recent searches, favourites, and the ability to add special instructions such as “no onions.” These needs directly shape the user stories and requirements below.

2.3 User Stories

1. As a Customer, I want browse the complete menu with prices and availability, so that I can choose food before arriving.
2. As a Customer, I want search for a dish by name, so that I can find food quickly.
3. As a Customer, I want filter the menu by category, so that I can focus on the type of food I want.
4. As a Customer, I want save favourite dishes, so that I can quickly find items I like.
5. As a Customer, I want reserve a table for a chosen date and time, so that I can avoid waiting at the restaurant.
6. As a Customer, I want add notes such as no onions to a cart item, so that my order matches my preference.
7. As a Customer, I want apply a valid promo code, so that I can see the discount before ordering.
8. As a Customer, I want see a full order summary, so that I understand subtotal, charges, tax, discount, and total.
9. As a Customer, I want track my order status, so that I know when the food is being prepared and when it is ready.
10. As a Customer, I want switch between light and dark themes, so that I can use the app comfortably.
11. As a Restaurant Manager, I want view incoming orders, so that I can prepare and update them efficiently.
12. As a Restaurant Manager, I want accept or decline table reservations, so that I can control table availability.
13. As a Restaurant Manager, I want add items, edit prices, and toggle menu availability, so that the customer menu stays current.
14. As a Restaurant Manager, I want change an order status manually, so that customers can see the latest status.

3. Functional Requirements

3.1 Authentication

- FR-01.** The system shall allow the user to switch between Login and Signup modes on one screen.
- FR-02.** The system shall validate email format, password length of at least eight characters, at least one digit, and matching confirmation password during signup.
- FR-03.** The system shall authenticate login credentials against the local mock users data set after a simulated one-second delay.
- FR-04.** The system shall route a successful Customer login to the customer menu and a successful Manager login to the manager dashboard.
- FR-05.** The system shall store the authenticated user in AuthContext and shall provide logout that resets navigation to Login.

3.2 Menu

- FR-06.** The system shall load at least fifteen menu items across Starters, Mains, Desserts, and Drinks from local mock data.
- FR-07.** The system shall display menu items using FlatList and show name, description, price, image, category, availability, and daily special status.
- FR-08.** The system shall disable the Add button and visually grey out unavailable menu items.
- FR-09.** The system shall allow the customer to filter items by category and sort by price low-to-high, price high-to-low, or name A-to-Z.
- FR-10.** The system shall allow the customer to toggle favourite menu items.

3.3 Search

- FR-11.** The system shall provide a search input that filters menu items by entered text.
- FR-12.** The system shall debounce search input by 400 milliseconds before applying the query.
- FR-13.** The system shall store and display the last five non-duplicate search terms as tappable suggestions.
- FR-14.** The system shall display a Back to Top control after the menu has been scrolled more than 300 pixels.
- FR-15.** The system shall display a friendly empty-state message when no menu item matches the search.

3.4 Cart

- FR-16.** The system shall allow an available menu item to be added to the cart.
- FR-17.** The system shall allow cart quantities to be incremented and decremented and shall remove an item when quantity reaches zero.
- FR-18.** The system shall allow the customer to add special instructions to each cart item.
- FR-19.** The system shall apply valid local promo codes and reject invalid codes with an error message.
- FR-20.** The system shall display a live cart item count badge.

3.5 Reservation

FR-21. The system shall offer hourly reservation time slots from 12:00 to 22:00.

FR-22. The system shall disable a time slot when no table with enough seats is available for the selected party size.

FR-23. The system shall validate that the reservation date is not in the past, party size is between 1 and 12, phone number matches 03XX-XXXXXXX, and booking time is at least one hour ahead.

FR-24. The system shall show a confirmation summary before creating a reservation and shall allow an existing reservation to be cancelled after confirmation.

3.6 Orders

FR-25. The system shall allow the customer to choose Dine-in or Takeaway before placing an order.

FR-26. The system shall calculate subtotal, 5 percent service charge, 15 percent sales tax, promo discount, and grand total.

FR-27. The system shall create a new order with id, items, total, type, Pending status, and timestamp.

FR-28. The system shall automatically demonstrate order progression from Pending to Preparing, Ready, and Served using timers.

FR-29. The system shall show a progress indicator and elapsed time on the Order Tracking screen.

3.7 Manager Dashboard

FR-30. The system shall display the Manager Dashboard only when the logged-in user role is manager.

FR-31. The system shall allow the manager to update incoming order status manually.

FR-32. The system shall allow the manager to accept or decline reservations.

FR-33. The system shall allow the manager to add menu items, edit prices, and toggle item availability.

FR-34. The system shall make manager menu changes immediately visible to customer menu screens through shared state.

FR-35. The system shall persist orders, reservations, and menu edits in AsyncStorage and restore them when the app starts.

4. Non-Functional Requirements

Area	Requirement
Usability	Controls shall use clear labels, readable text, feedback for validation errors, loading indicators, disabled states, and confirmation messages. Core customer tasks should be reachable with a small number of taps.
Performance	Mock menu loading shall complete after the specified simulated delay; FlatList shall be used for list rendering; filtering/sorting totals shall use memoization where required; scrolling should remain responsive for the provided data set.
Responsiveness	Layouts shall adapt to common phone screen sizes, use Safe Area handling, avoid clipped content, and support portrait use in Expo Go.
Maintainability	Source code shall be organized into components, screens, context, reducers, hooks, data, and navigation. Reusable

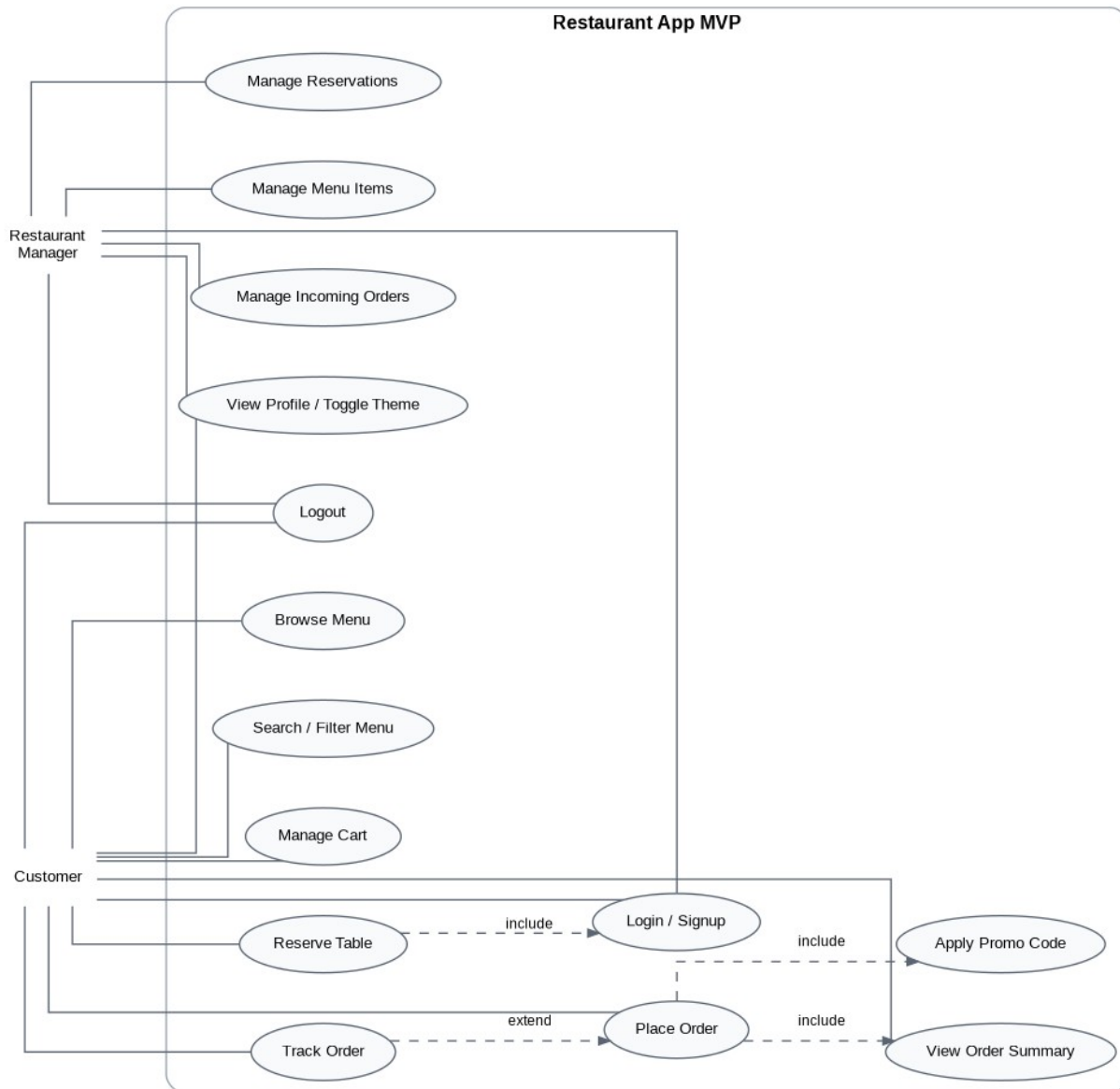
	logic shall be extracted into components and custom hooks.
Data Handling	No backend shall be required. Initial data shall be stored in local JavaScript/JSON mock files and working state shall be maintained in React state, Context, reducers, and AsyncStorage where specified.
Reliability	Timers and intervals shall be cleared during effect cleanup to prevent updates after screen unmount. Invalid forms shall not create records.
Accessibility	Buttons and input fields shall have descriptive text and sufficient touch areas; colour alone shall not be the only status indicator.
Compatibility	The project shall run as a React Native Expo application and be demonstrable in Expo Go with the dependencies documented in README.md.

5. Client-Side Data Model (Mock Data)

Data Set Name	Fields	Description
Users	id, fullName, email, password, role	Mock customer and manager accounts used for authentication and role-based navigation.
Categories	id, name	Menu group labels such as Starters, Mains, Desserts, and Drinks.
MenuItems	id, name, description, price, category, image, isSpecial, isAvailable	All menu items displayed to the customer and editable by the manager.
Tables	id, number, seats, isAvailable	Restaurant tables and their seating capacity used for reservation availability.
Reservations	id, customerId, date, time, partySize, tableId, phone, status	Bookings created by customers and accepted, declined, or cancelled.
Orders	id, customerId, items, total, type, tableId/pickupTime, status, timestamp	Placed orders and their tracking state.
PromoCodes	code, discountPercent	Local valid promo codes such as WELCOME10 and FEAST20.
Favourites	customerId, menuItemIds	Local list of customer favourite menu item ids.
SearchHistory	customerId, terms	Up to five recent non-duplicate search terms for suggestions.

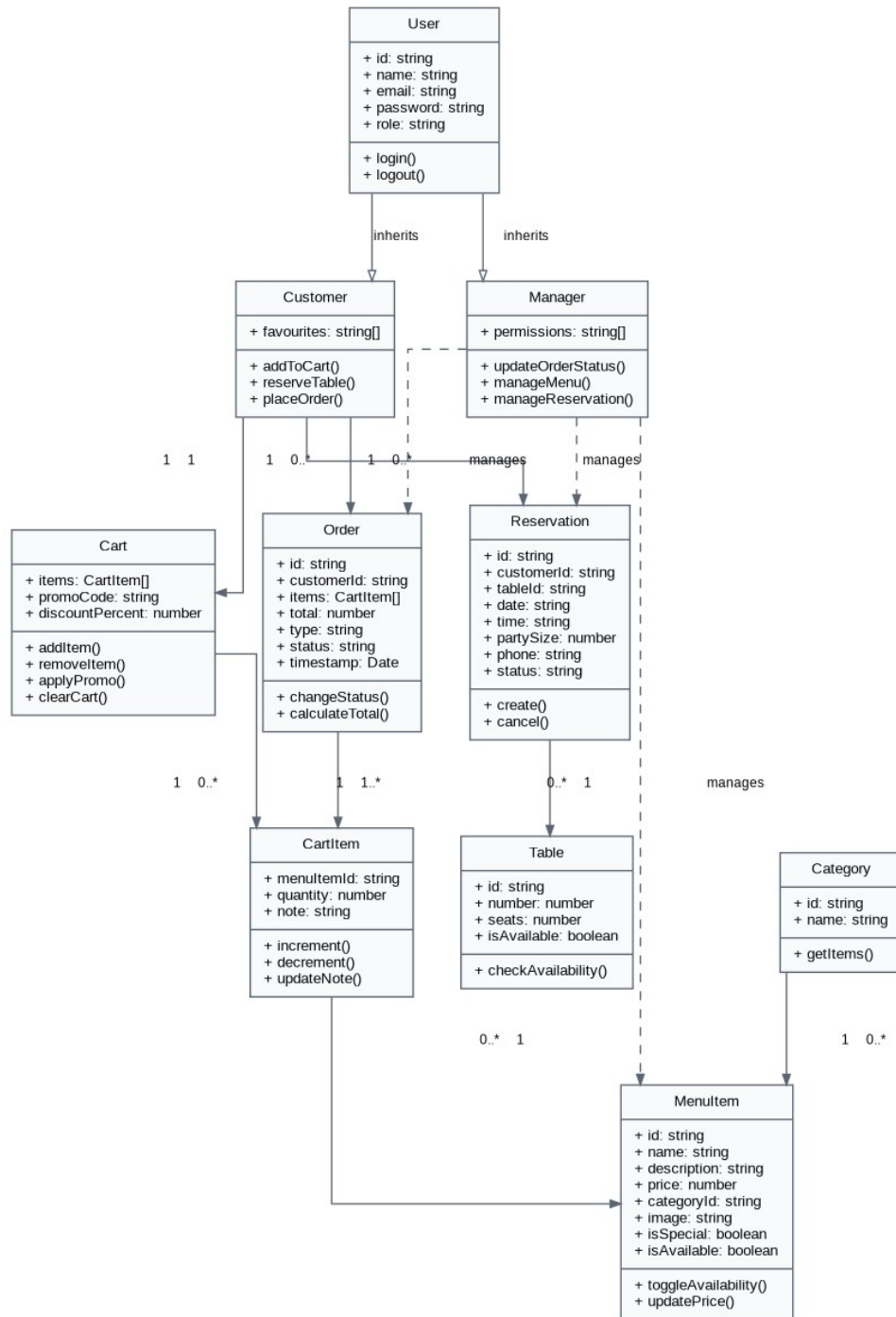
6. UML Diagrams

Figure 1. Use Case Diagram

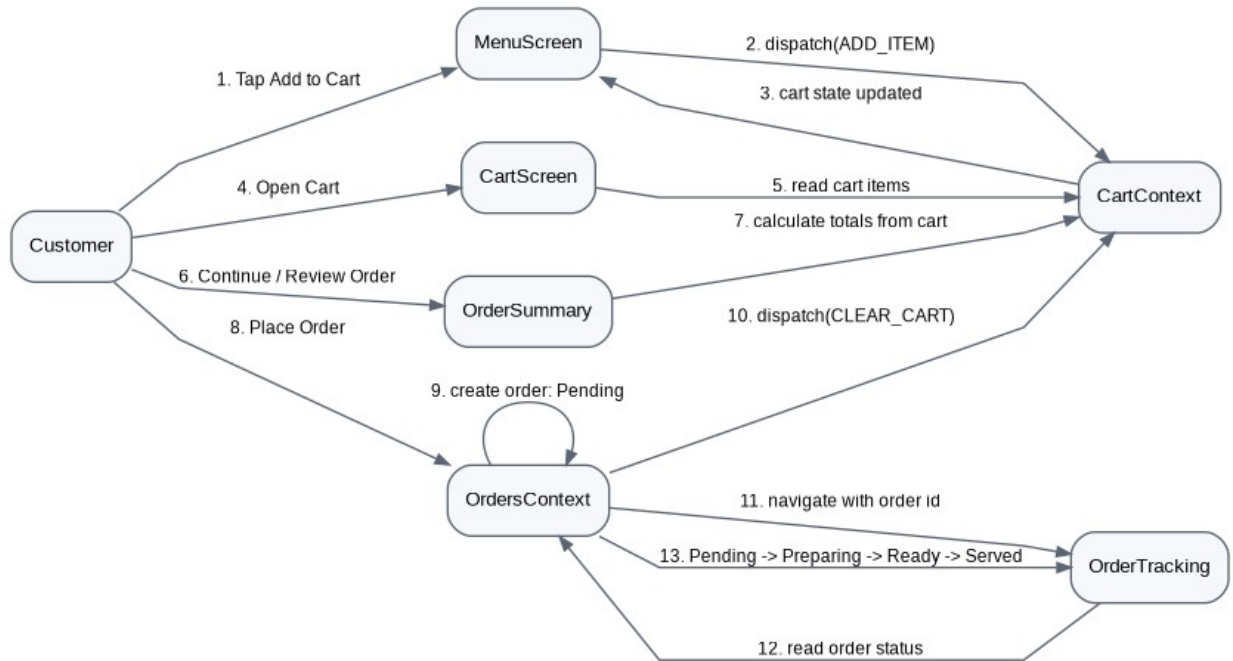


The use case diagram shows the two actors, Customer and Restaurant Manager, and the main actions available to each role. It includes customer activities such as browsing the menu, managing the cart, reserving a table, placing and tracking an order, and manager activities such as managing orders, reservations, and menu items. Include and extend relationships show dependencies such as Place Order including Order Summary and Promo Code handling.

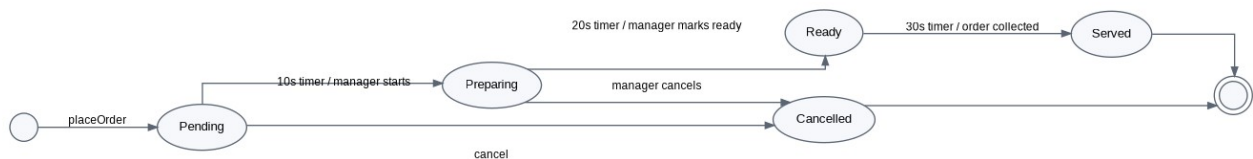
Figure 2. Class Diagram



The class diagram models the major client-side entities. **User** is the parent of **Customer** and **Manager**. **Categories** contain **MenuItems**; a **Customer** owns a **Cart** containing **CartItem**s; **Orders** contain cart items; and **Reservations** connect customers to **Tables**. **Manager** relationships indicate the management actions performed through the dashboard.

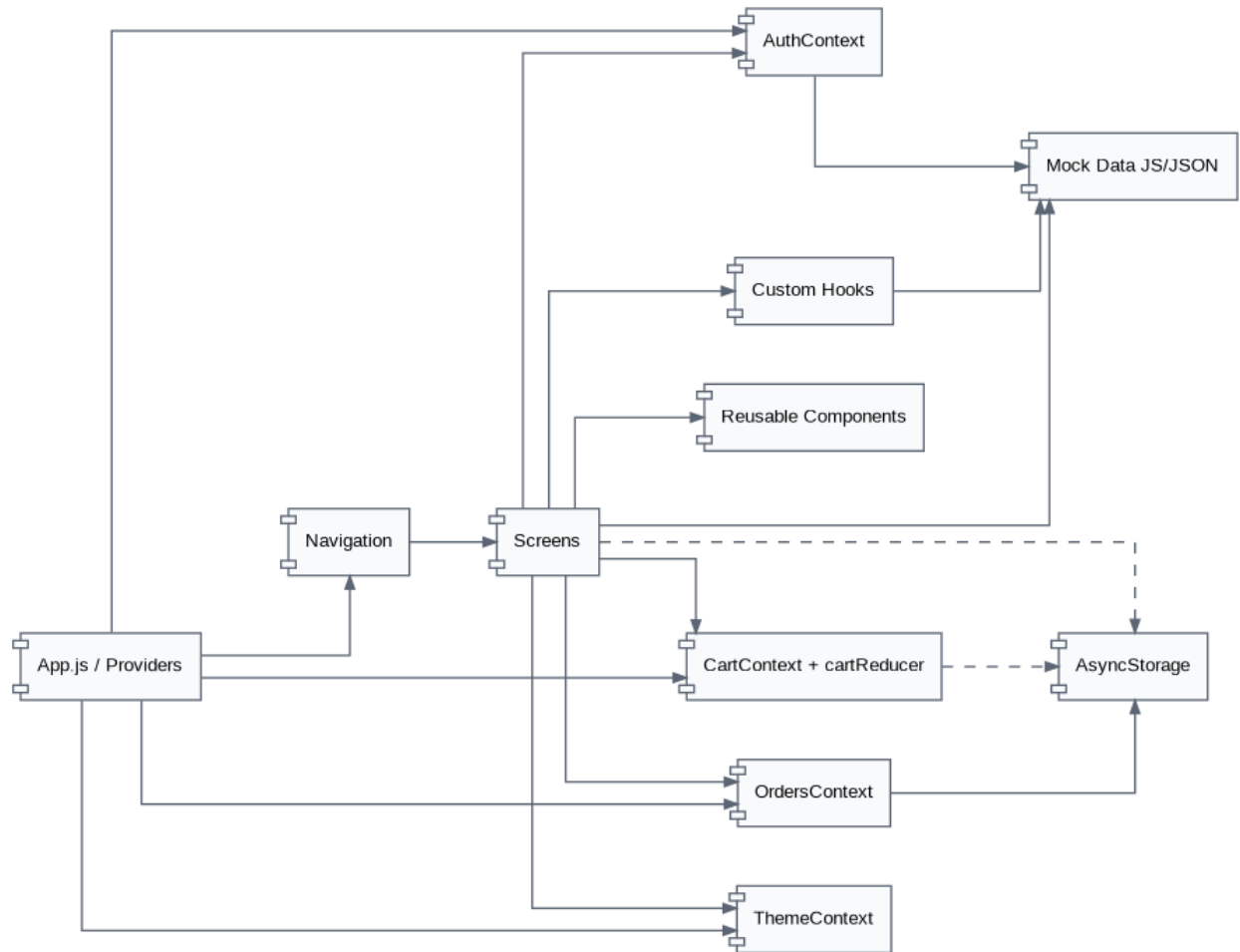
Figure 3. Sequence Diagram - Customer Places an Order

The sequence begins when the customer taps Add to Cart, continues through CartContext and the Cart screen, calculates totals in Order Summary, creates a Pending order through OrdersContext, clears the cart, and finally navigates to Order Tracking where the status progresses.

Figure 4. State Machine Diagram - Order Lifecycle

The state machine represents the order lifecycle required by the MVP. A placed order starts as Pending, moves to Preparing, then Ready, and finally Served. A Pending or Preparing order can move to Cancelled. Timer events and manager actions can trigger the transitions.

Figure 5. Component Diagram



The component diagram shows how App.js composes providers and navigation, how screens depend on shared Context providers and reusable components, how custom hooks use local mock data, and how AsyncStorage provides local persistence for selected state.

7. MVP Frontend Development (React Native)

7.1 Login and Signup Screen

Item	Details
Purpose	Allow users to authenticate or create a local account and select Customer or Manager role.
UI Elements	Login/Signup toggle, full name, email, password, confirm password, role selector, show-password control, validation messages, ActivityIndicator, submit button.
Navigation Entry / Exit	Entry: app start or logout. Exit: Menu for Customer, Dashboard for Manager.
Local Data Used	Users mock array.
Hooks and Why	useState for mode, controlled inputs, errors, password visibility, and isSubmitting; later refactored with useForm custom hook and AuthContext.

7.2 Menu Browsing Screen

Item	Details
Purpose	Display restaurant items, category filters, special items, availability, favourites, and add-to-cart actions.
UI Elements	Category chips, FlatList cards, Daily Special badge, favourite heart, Add button, pull-to-refresh, loading/error/Retry UI.
Navigation Entry / Exit	Entry: customer login or Menu tab. Exit: Cart, Profile, Reservation, item actions.
Local Data Used	Categories and MenuItem.s.
Hooks and Why	useState for loading/filter UI; useEffect for simulated fetch, header update, cleanup; useMemo for final derived filter/search/sort; useCallback for stable handlers.

7.3 Search and Scroll Controls

Item	Details
Purpose	Help customers quickly locate menu items and move around a long list.
UI Elements	Search TextInput, search icon, clear button, recent searches, render counter label, Back to Top floating button, empty state.
Navigation Entry / Exit	Entry: part of Menu screen. Exit: selecting a result or clearing search.
Local Data Used	MenuItem.s and SearchHistory.
Hooks and Why	useRef for TextInput, FlatList, timeout/render counter/previous query; later useDebounce replaces manual debounce. useState controls visible query-related UI.

7.4 Profile and Theme Screen

Item	Details
Purpose	Show logged-in user information and allow global theme switching and logout.
UI Elements	Name, email, role, light/dark switch, logout button.
Navigation Entry / Exit	Entry: Profile tab. Exit: other tabs or Login after logout.
Local Data Used	Authenticated User and theme palettes.
Hooks and Why	useContext through useAuth and useTheme to access global auth/theme values.

7.5 Cart Screen

Item	Details
Purpose	Manage selected dishes and instructions before checkout.
UI Elements	Cart item list, quantity stepper, remove button, note field, promo input/apply/remove, totals preview, checkout button.
Navigation Entry / Exit	Entry: Cart tab or Add-to-Cart flow. Exit: Menu or Order Summary.
Local Data Used	Cart items and PromoCodes.
Hooks and Why	useReducer through CartContext for related cart state transitions; useContext via useCart.

7.6 Order Summary Screen

Item	Details
Purpose	Show final price calculation before an order is placed.
UI Elements	Item summary, subtotal, service charge, tax, discount, grand total, Dine-in/Takeaway choice, continue/place-order control.
Navigation Entry / Exit	Entry: Cart. Exit: Order Tracking after order placement or back to Cart.
Local Data Used	Cart state and selected order type/table/pickup time.

Hooks and Why	useMemo computes subtotal, service charge, tax, discount, and grand total only when cart items/discount change.
---------------	---

7.7 Table Reservation Screen

Item	Details
Purpose	Create and cancel local table reservations with availability checking.
UI Elements	Date, time slots, party size, table/availability result, phone, confirmation modal, My Reservations list, cancel control.
Navigation Entry / Exit	Entry: Reservation tab. Exit: tab navigation or after booking confirmation.
Local Data Used	Tables and Reservations.
Hooks and Why	useReservation contains business logic; useForm handles reusable form state/validation; useDebounce is reused where needed.

7.8 Order Tracking and Manager Dashboard

Item	Details
Purpose	Track customer orders and provide manager controls over orders, reservations, and menu items.
UI Elements	Tracking progress steps and elapsed timer; manager tabs for Incoming Orders, Reservations, and Menu Management.
Navigation Entry / Exit	Entry: after Place Order for customer; manager login for dashboard. Exit: tabs/profile/logout.
Local Data Used	Orders, Reservations, MenuItem persisted in AsyncStorage.
Hooks and Why	useEffect for timers, cleanup, AsyncStorage load/save; OrdersContext with useReducer integrates global order state.

Appendix A: Required Implementation Notes

A.1 Filtering useEffect dependency note (Q4)

If the filtering effect has an empty dependency array, it runs only once after the component first mounts. Changes to selectedCategory or menuItems will not cause the effect to run again, so filteredItems can become stale and the UI will not update when the user selects another category or when new menu data is loaded. The effect must include selectedCategory and menuItems as dependencies when state is used for the derived list. In the later optimized version, the derived filtered/sorted list should instead be computed with useMemo.

A.2 Context vs prop drilling (Q6)

Auth and theme data are needed by many screens at different levels of the navigation tree. Context is suitable because it lets components read the current user or theme directly without passing the same props through intermediate screens that do not need them. It also gives login, logout, and theme switching one shared source of truth. A drawback is that consumers can re-render when the provided context value changes, so context values should be designed carefully.

A.3 useReducer vs useState for Cart (Q7)

The cart has several related transitions such as add, remove, increment, decrement, update note, clear cart, apply promo, and remove promo. useReducer keeps these state changes in one pure function and makes each transition explicit and testable. useState would be enough for a very small cart with only one or two independent values, but it becomes harder to maintain when many actions must update related state consistently.

A.4 Reducer Test Cases (Q7)

Action	Initial State	Expected State
ADD_ITEM	items=[]	Selected item exists with quantity=1.
INCREMENT	item quantity=1	Same item quantity=2.
DECREMENT	item quantity=1	Item is removed when quantity reaches 0.
UPDATE_NOTE	note=""	Item note equals the entered instruction.
APPLY_PROMO WELCOME10	discountPercent=0	promoCode="WELCOME10", discountPercent=10.
REMOVE_PROMO	discountPercent=10	promoCode="", discountPercent=0.
CLEAR_CART	items contain products	items=[] and promo fields reset.

A.5 When not to use useMemo/useCallback (Q8)

useMemo and useCallback should not be added to every value or function automatically. They have their own bookkeeping cost and can make code harder to read. They are most useful when a calculation is expensive, a large derived list is recomputed often, or stable function references prevent unnecessary rendering of memoized children. For small calculations and ordinary event handlers that do not cause a measurable performance problem, normal values and functions are simpler. Optimization should be based on a clear reason or observed re-render issue rather than used by default.